Boosting I (AdaBoost)

Amartya Sanyal

amsa@di.ku.dk
Department of Computer Science
University of Copenhagen

Based on slides by Fabian Giesecke.

Today!

Image source: https://dnc1994.com/2016/05/rank-10-percent-in-first-kaggle-competition-en/

Table of Contents Overview

Model Selection

When the features are set, we can start training models. Kaggle competitions usually favor **tree-based models**:

- **Gradient Boosted Trees**
- Random Forest
- Extra Randomized Trees

The following models are slightly worse in terms of general performance, but are suitable as base models in ensemble learning (will be discussed later):

- SVM
- Linear Regression
- Logistic Regression
- Neural Networks

Note that this does not apply to computer vision competitions which are pretty much dominated by neural network models.

All these models are implemented in **Sklearn**.

Here I want to emphasize the greatness of **Xgboost**. The outstanding performance of gradient boosted trees and Xgboost's efficient implementation makes it very popular in Kaggle competitions. Nowadays almost every winner uses Xgboost in one way or another.

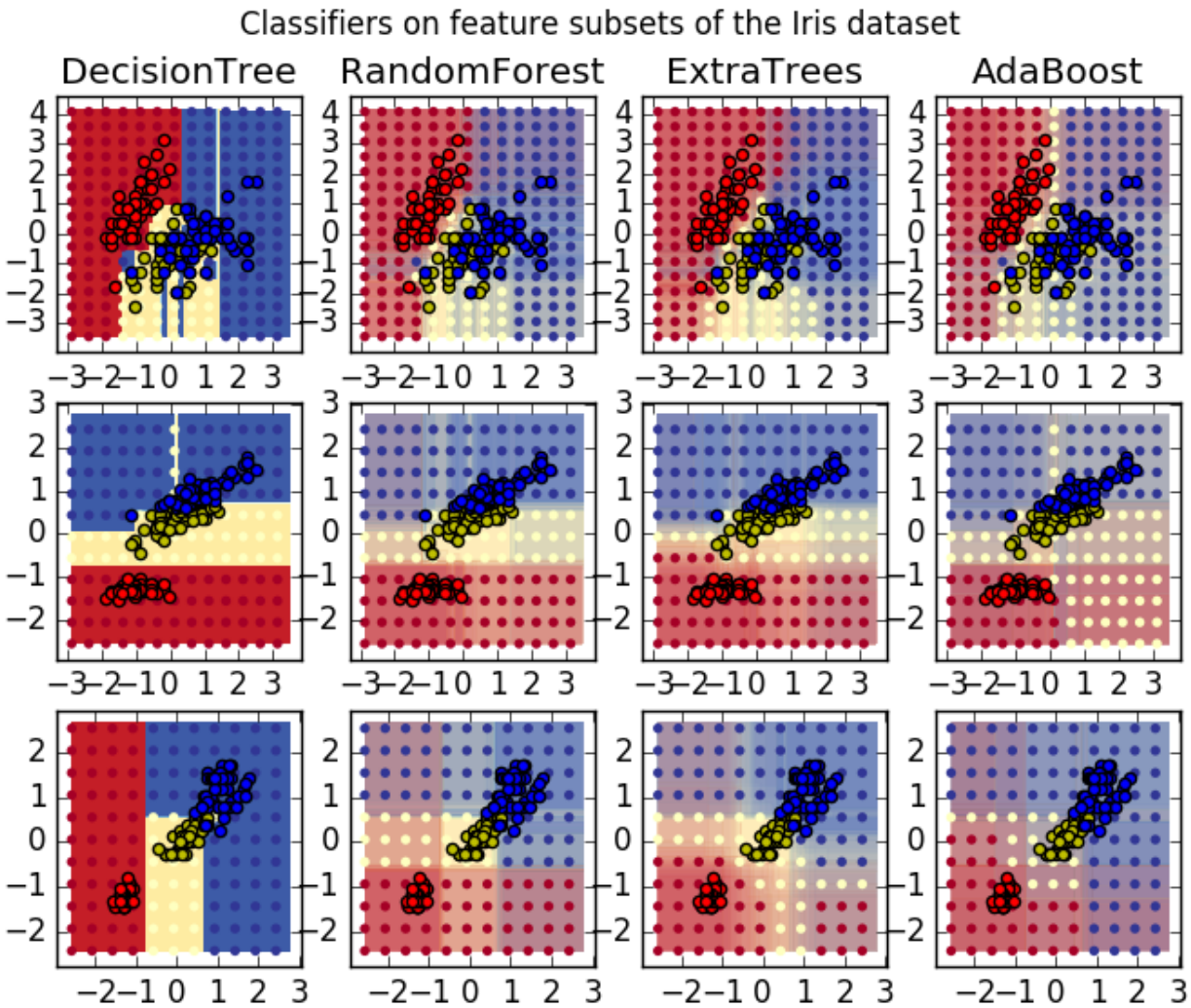
Updated on Oct 28th, 2016: Recently Microsoft open sourced **LightGBM**, a potentially better

Outline

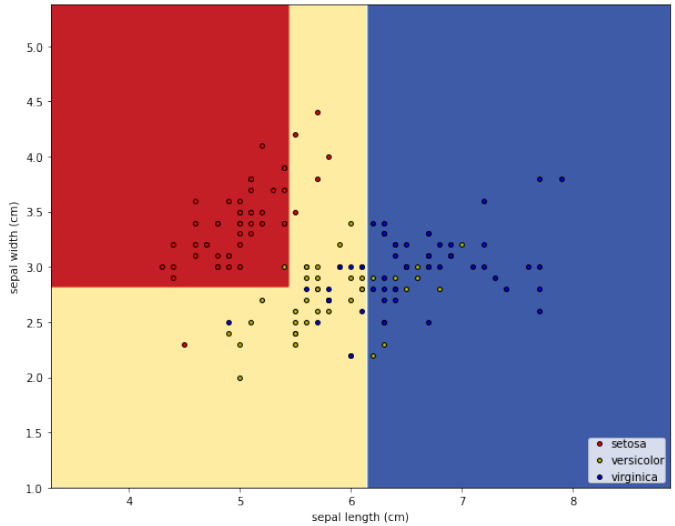
Outline

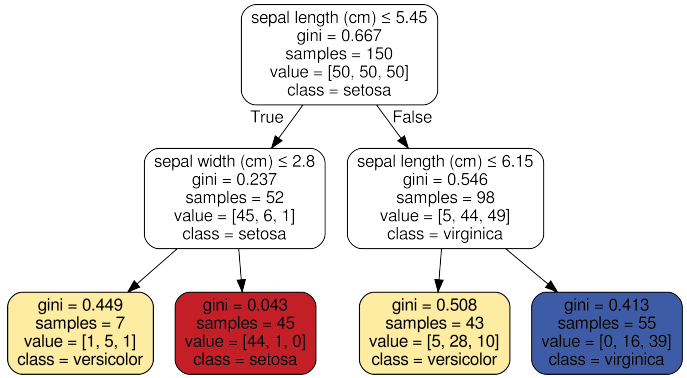
Tree Ensembles

Image source: https://scikit-learn.org/stable/modules/ensemble.html



Decision Trees





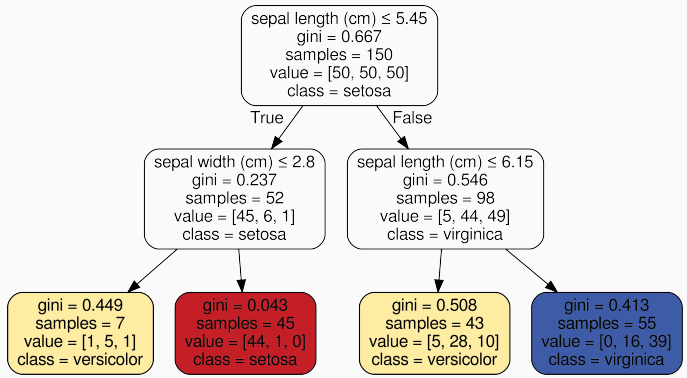
Key Facts

- Tree-based models are simple yet powerful models and are human interpretable.
- Tree-based methods partition the feature space into regions and assign a simple model (e.g., a constant) to each region.
- Several tree-based methods exist. We will focus on classification and regression trees (CARTs), which are binary trees.

Classification and Regression Trees (CARTs)

Constructing CARTs

- Every internal node is associated with one coordinate $i \in \{1, \dots, d\}$ and a threshold θ.
- At each inner node, the associated data $S = \{(\mathbf{x}_1, y_1), \dots\}$ at that node are split into $L_{i,\theta}$ and $R_{i,\theta}$ such that the information gain



$$G_{i,\theta}(S) = Q(S) - \frac{|L_{i,\theta}|}{|S|} Q(L_{i,\theta}) - \frac{|R_{i,\theta}|}{|S|} Q(R_{i,\theta})$$

  is maximized, where Q is some impurity measure.
- If the number $|S|$ of points at a node is smaller than a user-defined value or if S is pure (i.e., all points have the same label), then the node becomes a leaf (further stopping rules exist).
- After its construction, the tree can also be pruned (to avoid overfitting).

Classification and Regression Trees (CARTs)

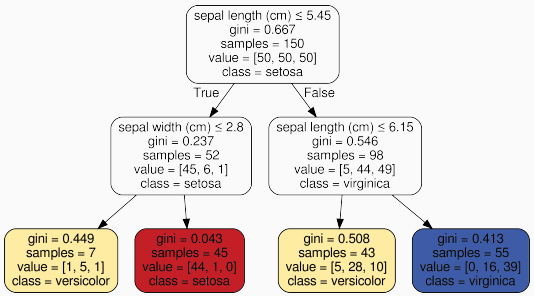
Recursive Tree Construction

Procedure: BuildTree(S,m)

Require: $S = \{(\mathbf{x}_1, y_1), \ldots\}$ and number m.

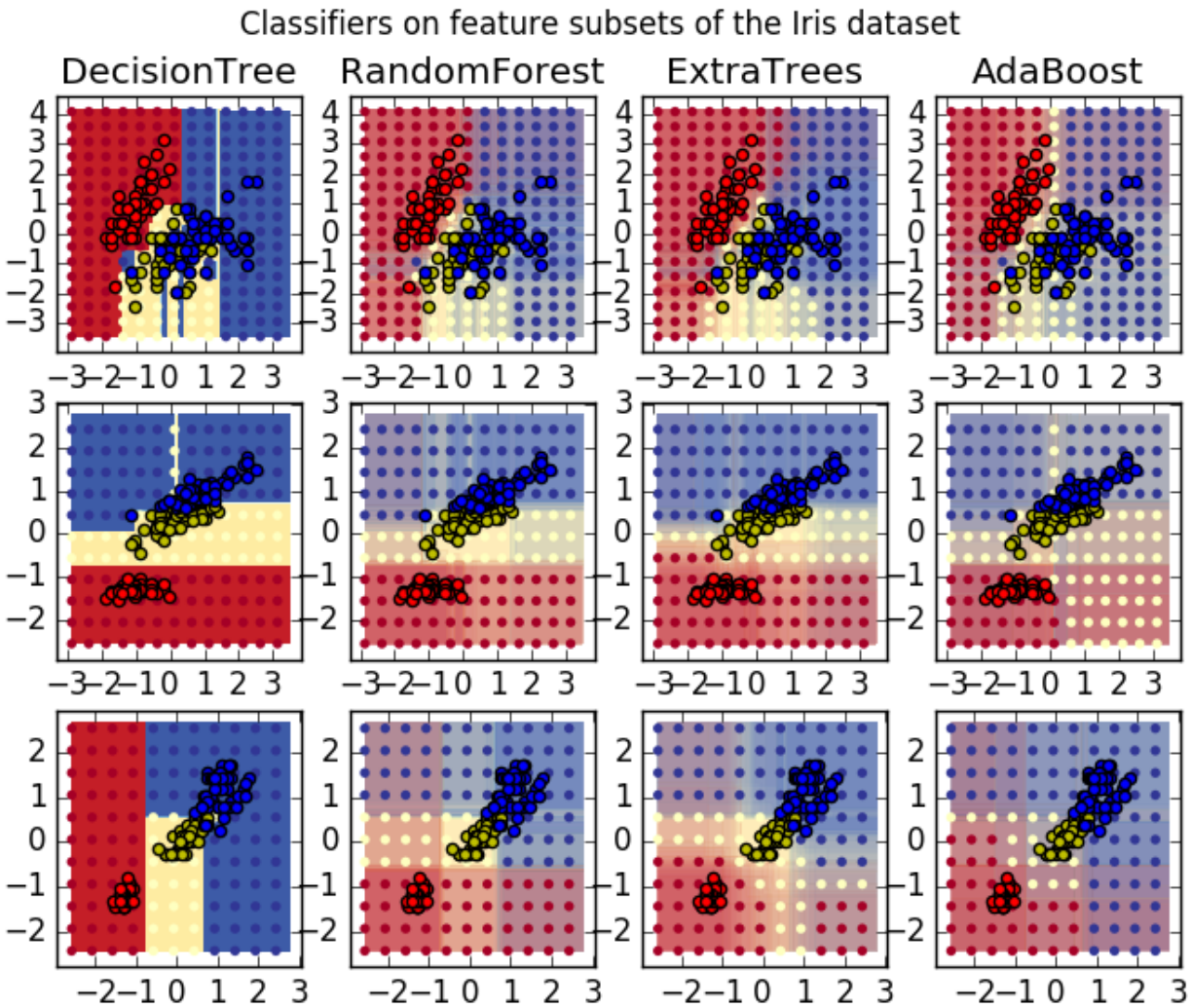
Ensure: Tree $\mathcal{T}$ built for the data instances in S.

1: **if** $|S| \leq m$ **then**
2: **return** leaf node storing the labels $\{y_1, \ldots, y_{|S|}\}$
3: **end if**
4: **if** $y_i = y_j$ for all $(\mathbf{x}_i, y_i), (\mathbf{x}_j, y_j) \in S$ **then**
5: **return** leaf node storing the labels $\{y_1, \ldots, y_{|S|}\}$
6: **end if**
7: Find $(i^*, \theta^*) = \text{argmax}_{i,\theta}\, G_{i,\theta}(S)$
8: $\mathcal{T}_l = \text{BuildTree}(L_{i^*,\theta^*}, \text{m})$
9: $\mathcal{T}_r = \text{BuildTree}(R_{i^*,\theta^*}, \text{m})$
10: Generate node that stores the splitting information (i^*, θ^*) and pointers to its subtrees $\mathcal{T}_l$ and $\mathcal{T}_r$. Let $\mathcal{T}$ denote the resulting tree.
11: **return** $\mathcal{T}$



Tree Ensembles

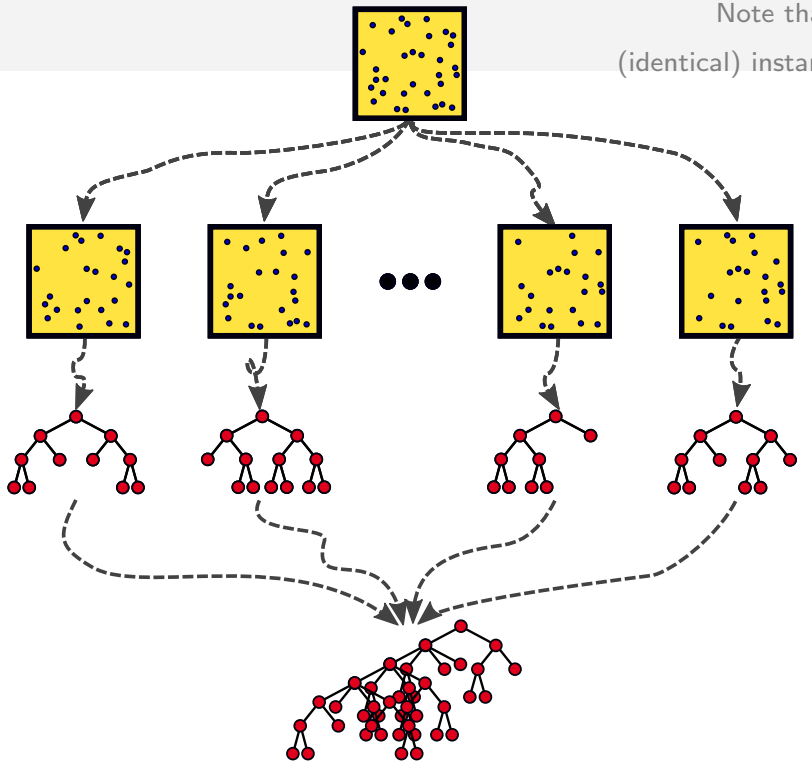
Image source: https://scikit-learn.org/stable/modules/ensemble.html



Original paper ([Bre01]): Leo Breimann. Random Forests, Machine Learning, 45(1):5–32, 2001.

General Idea

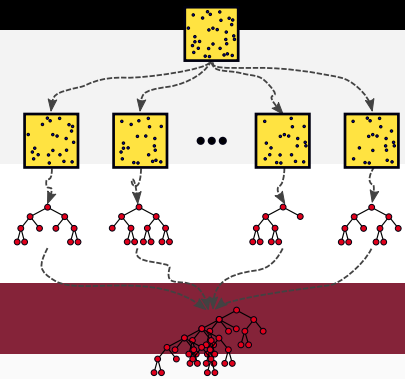
Note that we consider multisets, i.e., we allow multiple (identical) instances in the data T and bootstrap samples S_i.



Constructing Ensembles

1. Generate B different subsets $S_1, \ldots, S_B \subset T$ of your data $T \subset \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\} \subset \mathbb{R}^d \times \mathcal{Y}$. The subsets are called bootstrap samples.
2. For each subset S_b, build a decision tree $\mathcal{T}_b$.
3. The final prediction is obtained by aggregating the predictions made by the decision trees $\mathcal{T}_1, \ldots, \mathcal{T}_B$ (e.g., majority vote for classification).

Random Forests



Construction

Procedure: BUILDRF(S,m,f,B)

Require: $S = \{(\mathbf{x}_1, y_1), \ldots\}$, number m, number f of features to be tested per split, and number B of trees.

Ensure: Trees $\mathcal{T}_1, \ldots, \mathcal{T}_B$.

1: **for** $b = 1, \ldots, B$ **do**
2: Generate bootstrap sample S_b by drawing $|S|$ elements from S with replacement.
3: $\mathcal{T}_b = \text{BUILDRFTREE}(S_b, \text{m}, \text{f})$
4: **end for**
5: **return** $\mathcal{T}_1, \ldots, \mathcal{T}_B$

Predictions

A prediction $f(\mathbf{x})$ for a feature vector $\mathbf{x} \in \mathbb{R}^d$ is obtained by aggregating the predictions $\mathcal{T}_1(\mathbf{x}), \ldots, \mathcal{T}_B(\mathbf{x})$ made by the individual trees:

1. Regression: $f_{\text{RF}}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^{B} \mathcal{T}_b(\mathbf{x})$
2. Classification: $f_{\text{RF}}(\mathbf{x}) =$ majority vote among $\mathcal{T}_1, \ldots, \mathcal{T}_B$

Random Forests: Tree Construction

Recursive RF Tree Construction

Procedure: BuildRFTree(S,m,f)

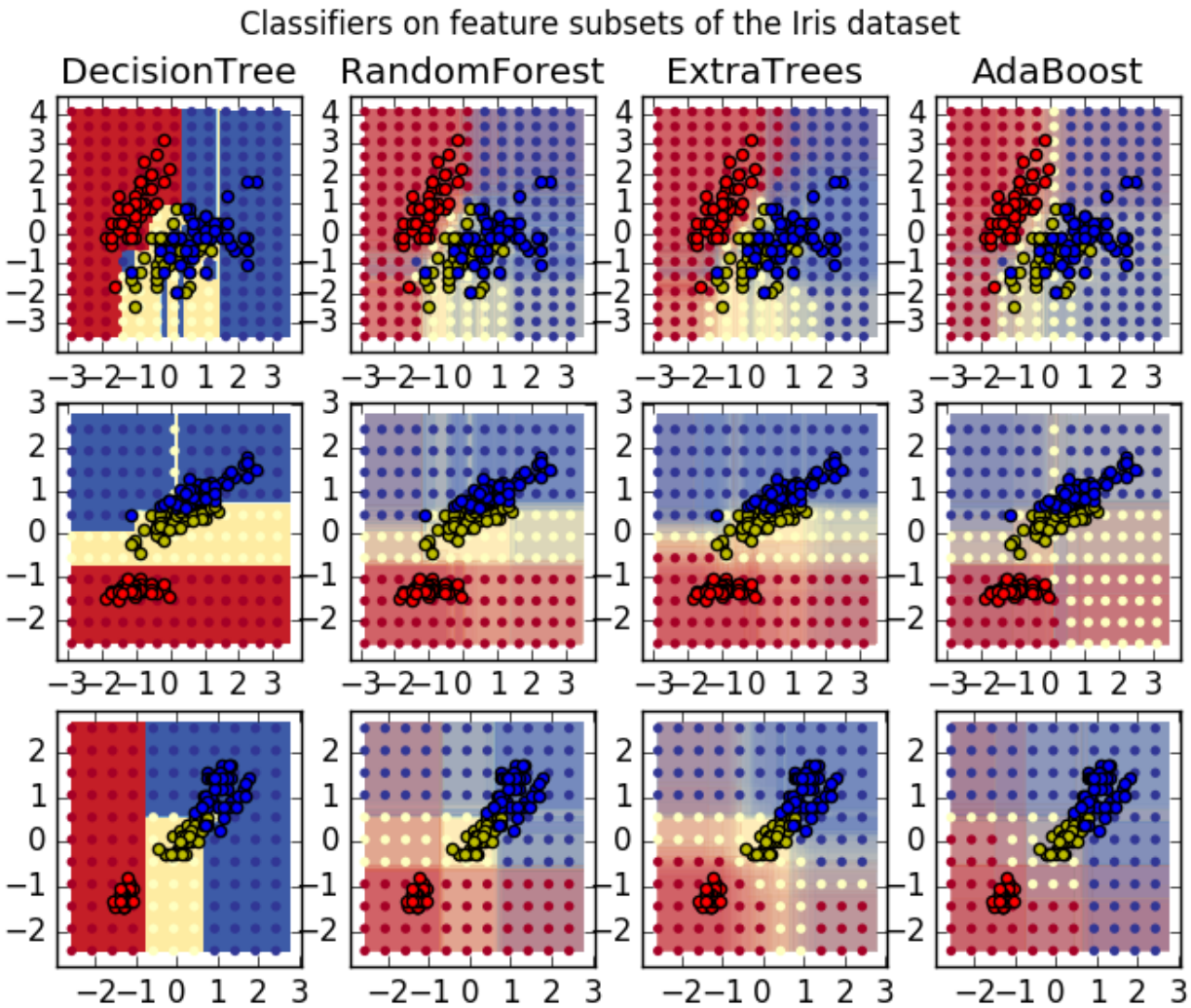
Require: $S = \{(\mathbf{x}_1, y_1), \ldots\}$, number m (usually m=1), and number f of features to be tested per split.

Ensure: Tree $\mathcal{T}$ built for the input instances in S.

1: **if** $|S| \leq m$ **then**
2: **return** leaf node storing the labels $\{y_1, \ldots, y_{|S|}\}$
3: **end if**
4: **if** $y_i = y_j$ for all $(\mathbf{x}_i, y_i), (\mathbf{x}_j, y_j) \in S$ **then**
5: **return** leaf node storing the labels $\{y_1, \ldots, y_{|S|}\}$
6: **end if**
7: Find $(i^*, \theta^*) = \text{argmax}_{i,\theta}\, G_{i,\theta}(S)$ for $i \in \{i_1, \ldots, i_f\} \subseteq \{1, \ldots, d\}$ (i.e., "only test f random dimensions")
8: $\mathcal{T}_l = \text{BuildRFTree}(L_{i^*,\theta^*}, \text{m}, \text{f})$
9: $\mathcal{T}_r = \text{BuildRFTree}(R_{i^*,\theta^*}, \text{m}, \text{f})$
10: Generate node ((i^*, θ^*) and pointers). Let $\mathcal{T}$ be the resulting tree.
11: **return** $\mathcal{T}$

Tree Ensembles

Image source: https://scikit-learn.org/stable/modules/ensemble.html



Original paper ([GEW06]): P. Geurts, D. Ernst, and L. Wehenkel. Extremely Randomized Trees, Machine Learning, 63(1):3–42, 2006.

Random Forest → Extra Trees

Extra Trees

Basically the same as random forests. Two small modifications:

1. Do not draw bootstrap samples.
2. Use random threshold $\hat{\theta} \in \left[\min_{(\mathbf{x},y)\in S} x_i, \max_{(\mathbf{x},y)\in S} x_i\right]$ per dimension i.

Why does this still work?! Why does this even work better in practice?

Answer: We still compute the qualities of the random splits and select the best-performing one. Also, we introduce more randomness, which is usually helpful in practice.

Good Implementation

Image source: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

Implementation Hacks!

The Scikit-Learn implementation is extremely efficient as long as the data/trees fit in main memory. The code is written in Cython (hence: C code). Various "hacks" are used:

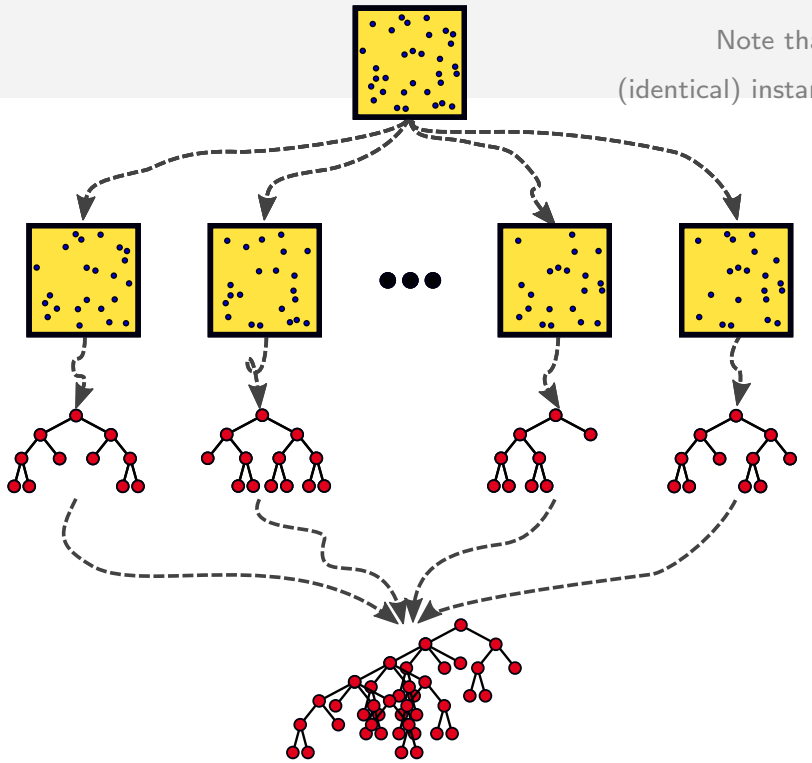
1. Trees & Arrays: Each tree is stored in an array (low-level, C-like)
2. Sorting: Optimized sorting algorithm (standard `qsort` $\rightarrow$ factor 2-3 slower)
3. Locally constant features: Keep track of "constant" features during construction.
4. Unique instances: If `bootstrap=True`, only use unique indices/patterns (only 63%) and assign weights to them $\rightarrow$ Speed-up of about 1.5!
5. Consecutive memory access: For evaluating $\mathcal{O}(S)$, prefetch data for dimension i
6. Sparse data: Make use of optimized computations for sparse data.
7. Random numbers: Use manual random number generator ...
8. Parallelization: For random forests, efficient parallelization over trees.
9. ...

In a nutshell: This implementation might easily be a factor of 10-100 faster than a "standard" implementation in C. Speed-up depends on the particular dataset ...

Outline

Bagging

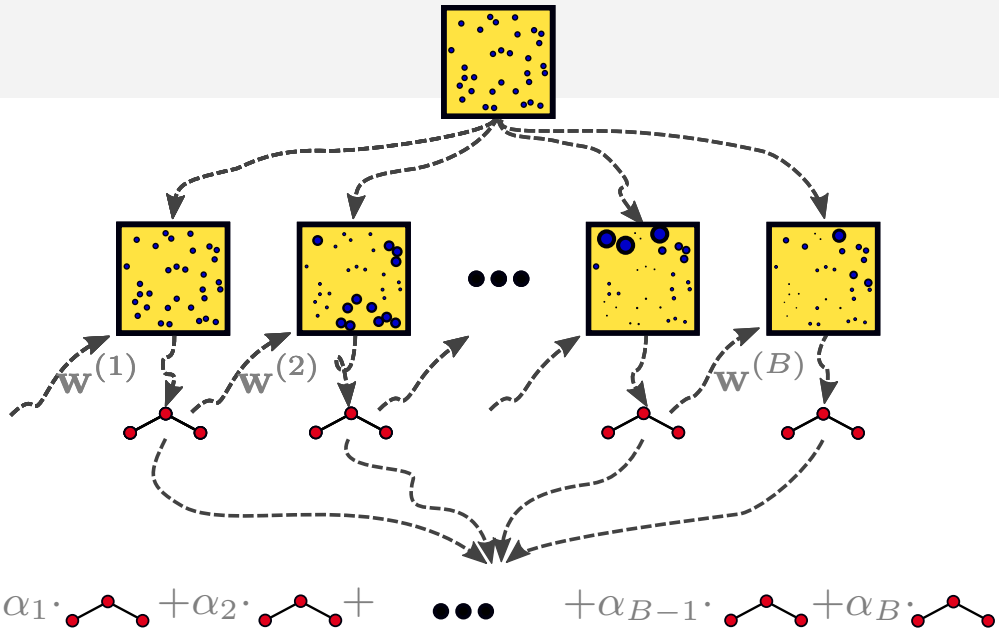
Note that we consider multisets, i.e., we allow multiple (identical) instances in the data T and bootstrap samples S_i.



Bootstrap Aggregating

1. Generate B bootstrap samples $S_1, \ldots, S_B$ from your training data $T = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\} \subset \mathbb{R}^d \times \mathcal{Y}$ (e.g., by sampling with replacement).
2. For each bootstrap sample S_b, build learner/model $\mathcal{T}_b$ (e.g., decision trees).
3. The final prediction is obtained by aggregating the predictions made by the learners $\mathcal{T}_1, \ldots, \mathcal{T}_B$ (e.g., majority vote for classification).

Boosting



Boosting

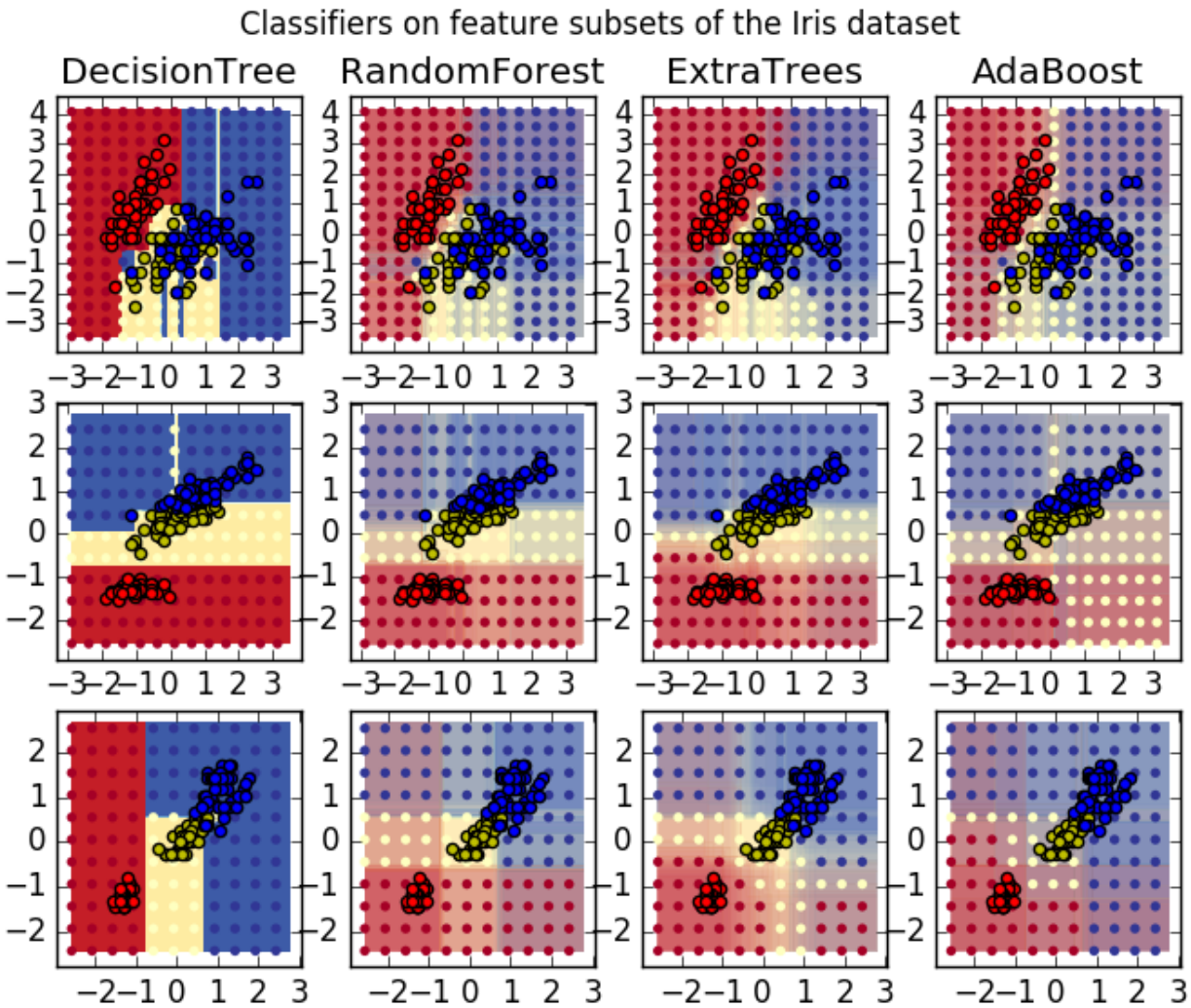
1. Given training data $T = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\} \subset \mathbb{R}^d \times \mathcal{Y}$ and weights $w_i^{(1)} = \frac{1}{n}$.
2. For $b = 1, \ldots, B$:
   - Train weak learner $\mathcal{T}_b$ based on training data T and weights $\mathbf{w}^{(b)}$.
     (e.g., construct i.i.d examples according to $\mathbf{w}^{(b)}$ or incorporate weights into fitting)
   - Increase weights $w_i^{(b)}$ for instances misclassified by $\mathcal{T}_b$; decrease weights $w_i^{(b)}$ for correctly classified instances. Normalize weights; this yields a new vector $\mathbf{w}^{(b+1)}$.
3. The final prediction is obtained via a weighted aggregation of the predictions made by the learners $\mathcal{T}_1, \ldots, \mathcal{T}_B$ (e.g., weighted majority vote).

weak learner = model slightly better than guessing (e.g., decision stump)

Outline

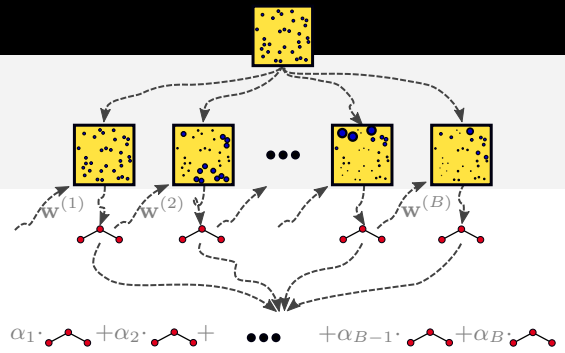
Tree Ensembles

Image source: https://scikit-learn.org/stable/modules/ensemble.html



Original paper ([FS96]): Yoav Freund and Robert E. Schapire. Experiments With a New Boosting Algorithm, ICML, 1996.

AdaBoost



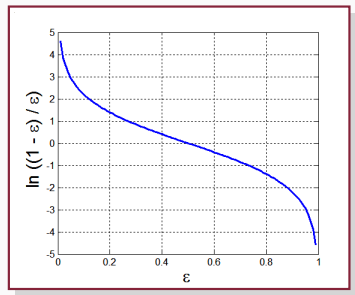
AdaBoost Algorithm

Require: Let $T = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\} \subset \mathbb{R}^d \times \{-1, 1\}$ and $B \in \mathbb{N}$

1: $\mathbf{w}^{(1)} = (\frac{1}{n}, \ldots, \frac{1}{n}) \in \mathbb{R}^n$
2: **for** $b = 1, \ldots, B$ **do**
3: $h_b = \text{FitWeakLearner}(T, \mathbf{w}^{(b)})$
4: Compute weighted classification error $\varepsilon_b = \sum_{j=1}^n w_j^{(b)} \mathbb{I}(h_b(\mathbf{x}_j) \neq y_j)$ of h_b
5: Ensure $\varepsilon_b < 0.5$ (e.g., by "inverting" classifier)
6: Compute importance $\alpha_b = \frac{1}{2} \ln\left(\frac{1-\varepsilon_b}{\varepsilon_b}\right)$ of classifier
7: For $i = 1, \ldots, n$, update weights via

$$w_i^{(b+1)} = \frac{w_i^{(b)} \exp(-\alpha_b y_i h_b(\mathbf{x}_i))}{\sum_{j=1}^n w_j^{(b)} \exp(-\alpha_b y_j h_b(\mathbf{x}_j))}$$

8: **end for**



Output hypothesis: $h(\mathbf{x}) = \text{sign}\left(\sum_{b=1}^B \alpha_b h_b(\mathbf{x})\right)$

Example: AdaBoost

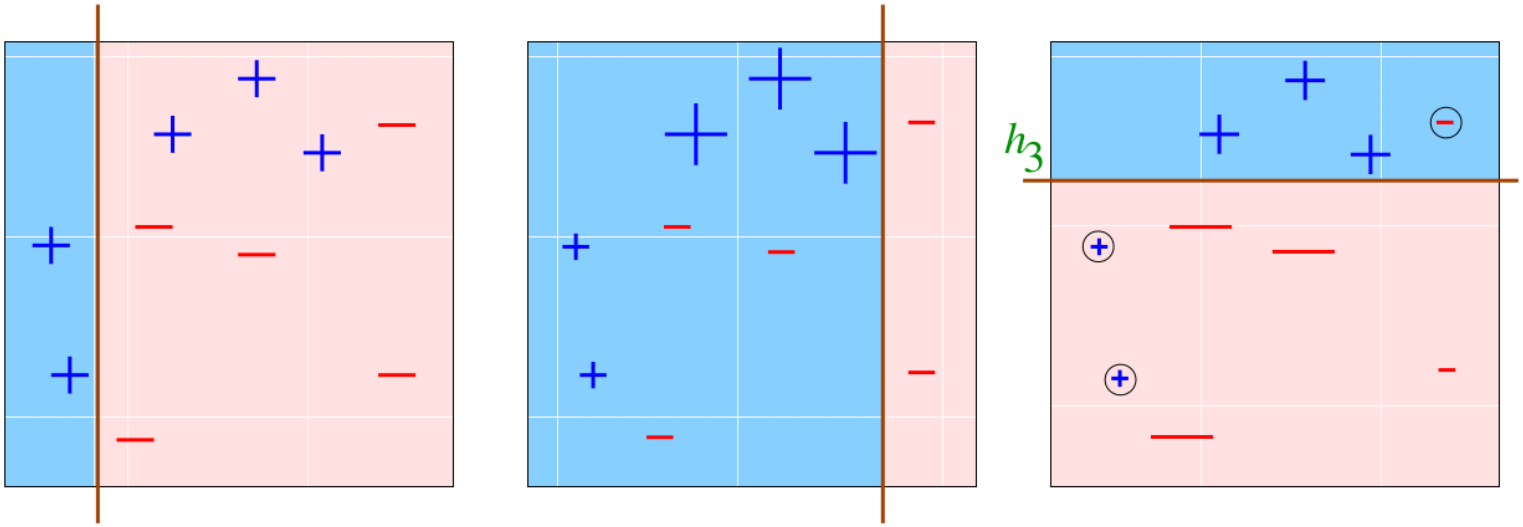


Image source: http://rob.schapire.net/

Example: AdaBoost

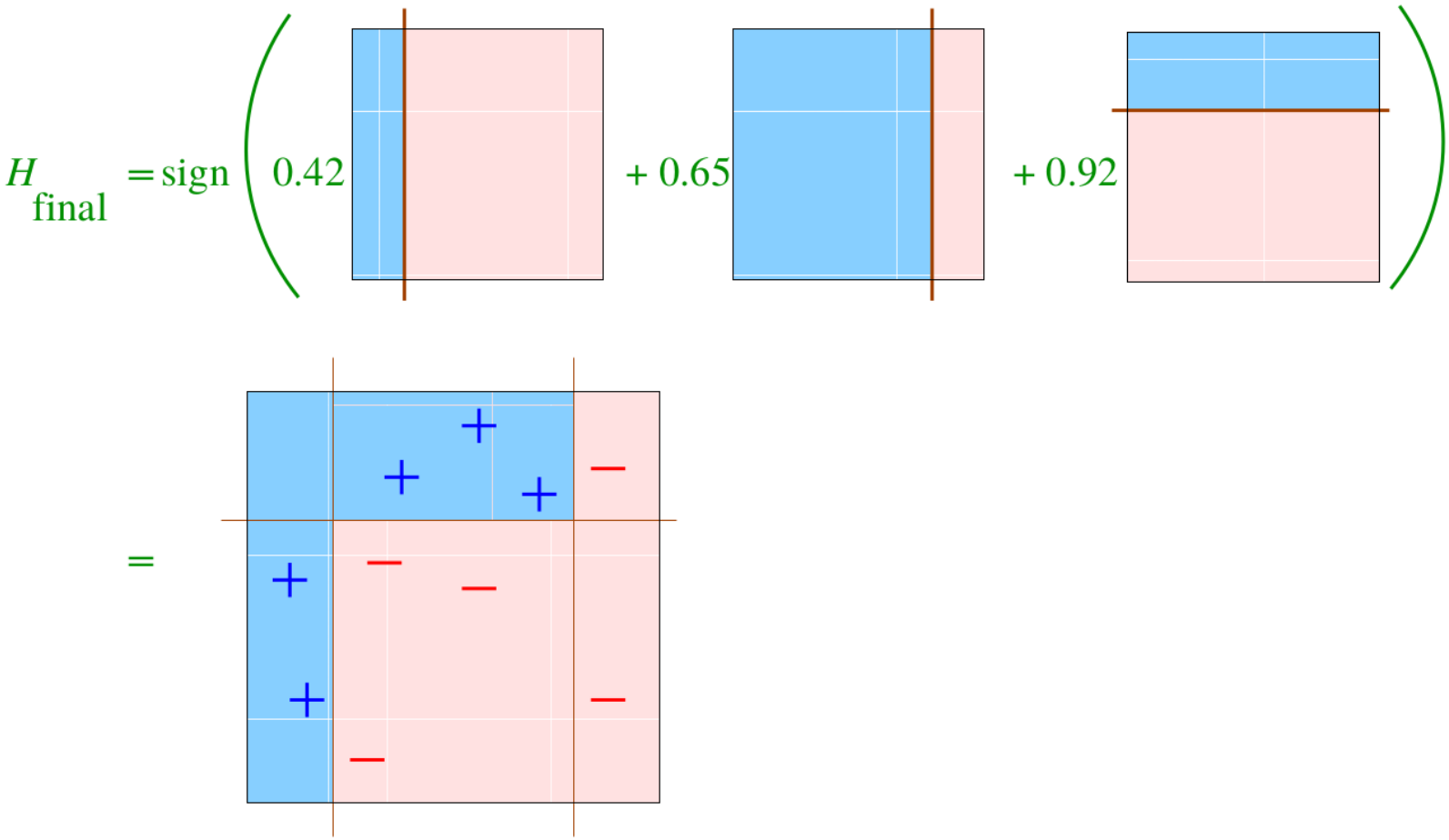
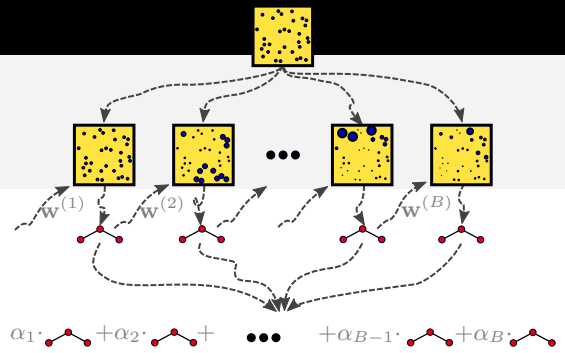


Image source: http://rob.schapire.net/

Recap: AdaBoost



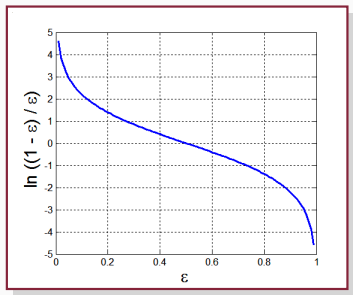
AdaBoost Algorithm

Require: Let $T = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\} \subset \mathbb{R}^d \times \{-1, 1\}$ and $B \in \mathbb{N}$

1: $\mathbf{w}^{(1)} = (\frac{1}{n}, \ldots, \frac{1}{n}) \in \mathbb{R}^n$
2: **for** $b = 1, \ldots, B$ **do**
3: $h_b = \textsc{FitWeakLearner}(T, \mathbf{w}^{(b)})$
4: Compute weighted classification error $\varepsilon_b = \sum_{j=1}^n w_j^{(b)} \mathbb{I}(h_b(\mathbf{x}_j) \neq y_j)$ of h_b
5: Ensure $\varepsilon_b < 0.5$ (e.g., by "inverting" classifier)
6: Compute importance $\alpha_b = \frac{1}{2} \ln\left(\frac{1-\varepsilon_b}{\varepsilon_b}\right)$ of classifier
7: For $i = 1, \ldots, n$, update weights via

$$w_i^{(b+1)} = \frac{w_i^{(b)} \exp(-\alpha_b y_i h_b(\mathbf{x}_i))}{\sum_{j=1}^n w_j^{(b)} \exp(-\alpha_b y_j h_b(\mathbf{x}_j))}$$

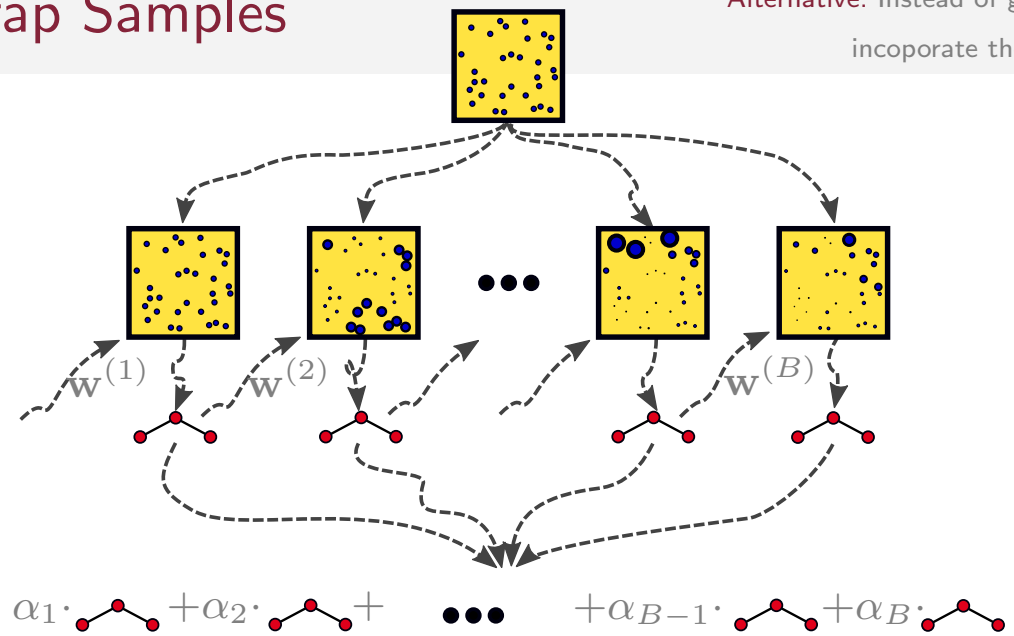
8: **end for**



Output hypothesis: $h(\mathbf{x}) = \text{sign}\left(\sum_{b=1}^B \alpha_b h_b(\mathbf{x})\right)$

Weighted Bootstrap Samples

Alternative: Instead of generating a bootstrap sample, directly incoporate the weights $w_i^{(b)}$ when generating h_b.



Generating Weighted Bootstrap Samples

- The weights sum up to 1, i.e., $\sum_{i=1}^{n} w_i^{(b)} = 1$
- Hence, they induce subintervals that partition the interval $[0, 1)$:

$$[0, w_1^{(b)}), [w_1^{(b)}, w_1^{(b)} + w_2^{(b)}), \ldots, [w_1^{(b)} + \ldots + w_{n-1}^{(b)}, 1)$$

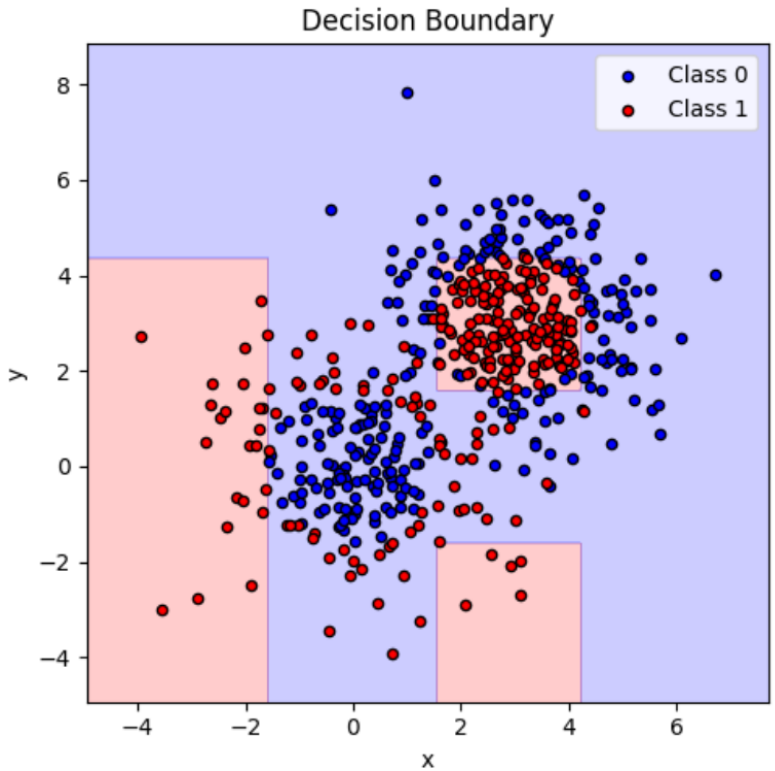
- To generate the bootstrap sample: Draw $z \sim \text{Uniform}(0, 1)$ and select the subinterval z falls into. Add corresponding instance to bootstrap sample (e.g., $(\mathbf{x}_1, y_1)$ if $z \in [0, w_1^{(b)})$); repeat n times. Fit weak learner h_b on bootstrap sample.

Demo Time!

jupyter Adaboost Last Checkpoint: a few seconds ago (autosaved) Logout

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (ipykernel)

Run Code

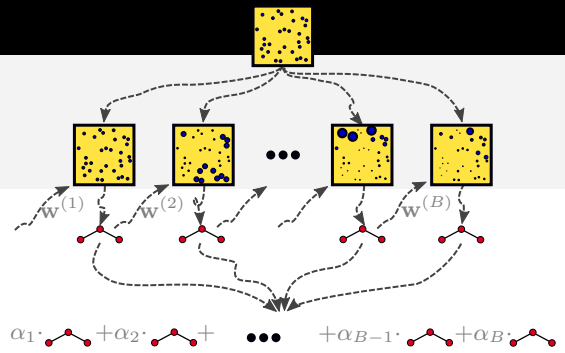


Finally, let us plot the decision surfaces and the tree structures of the individual trees. Note that the trees are built on modified training instances (weights!).

In [5]:
```
import graphviz
from sklearn import tree
from IPython.display import display

plot_colors = "br"
```

Recap: AdaBoost



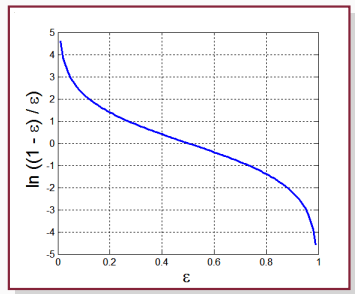
AdaBoost Algorithm

Require: Let $T = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\} \subset \mathbb{R}^d \times \{-1, 1\}$ and $B \in \mathbb{N}$

1: $\mathbf{w}^{(1)} = (\frac{1}{n}, \ldots, \frac{1}{n}) \in \mathbb{R}^n$
2: **for** $b = 1, \ldots, B$ **do**
3: $h_b = \text{FitWeakLearner}(T, \mathbf{w}^{(b)})$
4: Compute weighted classification error $\varepsilon_b = \sum_{j=1}^n w_j^{(b)} \mathbb{I}(h_b(\mathbf{x}_j) \neq y_j)$ of h_b
5: Ensure $\varepsilon_b < 0.5$ (e.g., by "inverting" classifier)
6: Compute importance $\alpha_b = \frac{1}{2} \ln\left(\frac{1-\varepsilon_b}{\varepsilon_b}\right)$ of classifier
7: For $i = 1, \ldots, n$, update weights via

$$w_i^{(b+1)} = \frac{w_i^{(b)} \exp(-\alpha_b y_i h_b(\mathbf{x}_i))}{\sum_{j=1}^n w_j^{(b)} \exp(-\alpha_b y_j h_b(\mathbf{x}_j))}$$

8: **end for**



Output hypothesis: $h(\mathbf{x}) = \text{sign}\left(\sum_{b=1}^B \alpha_b h_b(\mathbf{x})\right)$

Training Error

Theorem (Theorem 10.2 in [SSBD14])

Let T be a training set and assume that at each iteration of AdaBoost, the weak learner returns a hypothesis for which $\varepsilon_b \leq \frac{1}{2} - \gamma$ for some $\gamma > 0$. Then, the training error $L_T(h)$ of the output hypothesis h of AdaBoost is at most:

$$L_T(h) = \frac{1}{n} \sum_{i=1}^{n} \mathbb{I}(h(\mathbf{x}_i) \neq y_i) \leq \exp(-2\gamma^2 B)$$

Proof

For each b, denote $f_b := \sum_{p \leq b} \alpha_p h_p$. In addition, let

$$Z_b := \frac{1}{n} \sum_{i=1}^{n} \exp(-y_i f_b(\mathbf{x}_i)).$$

For the output hypothesis h, we have $\mathbb{I}(h(\mathbf{x}_i) \neq y_i) \leq \exp(-y_i h(\mathbf{x}_i))$, and, hence, $L_T(h) = L_T(f_B) \leq Z_B$. Thus, it is sufficient to show that $Z_B \leq \exp(-2\gamma^2 B)$.

Training Error

Proof (Continued)

Thus, it is sufficient to show that $Z_B \leq \exp(-2\gamma^2 B)$. Since $f_0 \equiv 0$, we have $Z_0 = 1$. Hence, we can rewrite Z_B as:

$$Z_B = \frac{Z_B}{Z_0} = \frac{Z_B}{Z_{B-1}} \cdot \frac{Z_{B-1}}{Z_{B-2}} \cdots \frac{Z_2}{Z_1} \cdot \frac{Z_1}{Z_0}$$

Hence, it suffices to show that $\frac{Z_{b+1}}{Z_b} \leq \exp(-2\gamma^2)$ for any b.

In the following, we will make use of the fact that, for any i and b, we have (see Assignment!):

$$w_i^{(b+1)} = \frac{\exp(-y_i f_b(\mathbf{x}_i))}{\sum_{j=1}^n \exp(-y_j f_b(\mathbf{x}_j))}$$

Training Error

Proof (Continued)

$$
\begin{aligned}
\frac{Z_{b+1}}{Z_b} &= \frac{\sum_{i=1}^n \exp(-y_i f_{b+1}(\mathbf{x}_i))}{\sum_{j=1}^n \exp(-y_j f_b(\mathbf{x}_j))} \\
&= \frac{\sum_{i=1}^n \exp\left(-y_i f_b(\mathbf{x}_i)\right) \exp\left(-y_i \alpha_{b+1} h_{b+1}(\mathbf{x}_i)\right)}{\sum_{j=1}^n \exp(-y_j f_b(\mathbf{x}_j))} \\
&= \sum_{i=1}^n w_i^{(b+1)} \exp(-y_i \alpha_{b+1} h_{b+1}(\mathbf{x}_i))) \\
&= \exp(-\alpha_{b+1}) \sum_{i: y_i h_{b+1}(\mathbf{x}_i)=1} w_i^{(b+1)} + \exp(\alpha_{b+1}) \sum_{i: y_i h_{b+1}(\mathbf{x}_i)=-1} w_i^{(b+1)} \\
&= \exp(-\alpha_{b+1})(1-\varepsilon_{b+1}) + \exp(\alpha_{b+1})\varepsilon_{b+1} \\
&= \sqrt{\frac{\varepsilon_{b+1}}{1-\varepsilon_{b+1}}}(1-\varepsilon_{b+1}) + \sqrt{\frac{1-\varepsilon_{b+1}}{\varepsilon_{b+1}}}\varepsilon_{b+1} \\
&= 2\sqrt{\varepsilon_{b+1}(1-\varepsilon_{b+1})}
\end{aligned}
$$

Training Error

Proof (Continued)

$$\begin{aligned}\frac{Z_{b+1}}{Z_b} &= 2\sqrt{\varepsilon_{b+1}(1-\varepsilon_{b+1})} \\ &\leq 2\sqrt{\left(\frac{1}{2}-\gamma\right)\left(\frac{1}{2}+\gamma\right)} = \sqrt{1-4\gamma^2}\end{aligned}$$

Here, the last line follows from the assumption that $\varepsilon_{b+1} \leq \frac{1}{2} - \gamma$ and the fact that the function $g(a) = a(1-a)$ is monotonically increasing in the interval $[0, \frac{1}{2}]$.

Finally, using $1 - a \leq \exp(-a)$, we have $\sqrt{1-4\gamma^2} \leq \exp(-4\gamma^2/2)$, which concludes the proof. □

Multi-Class Scenarios?

Multi-Class AdaBoost (SAMME)

- So far: Reset the weights in case $\varepsilon_b \geq 0.5$.
  - Problem: For many classes K, this might happen too often for "simple" classifiers!
  - Guessing: Error of about $1 - \frac{1}{K}$ (balanced dataset).
- Modification I: Only reset weights if $\varepsilon_b \geq 1 - \frac{1}{K}$
- Modification II: The importance α_b of a weak learner is now defined as

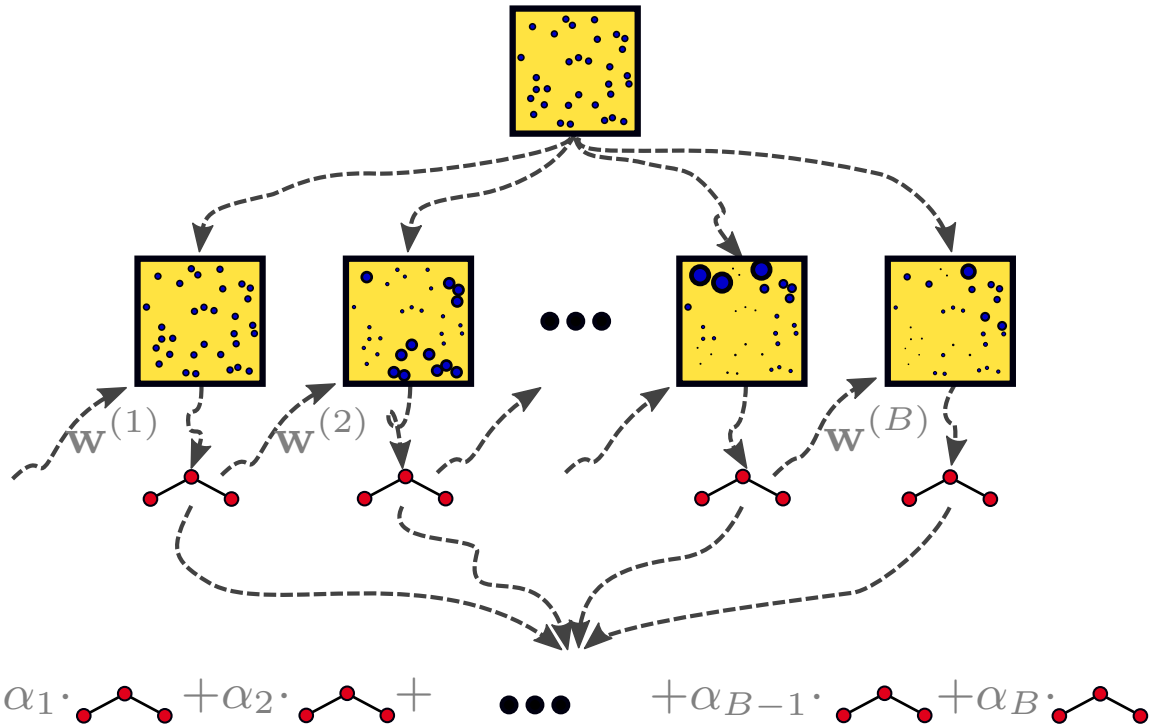
$$\alpha_b = \ln\left(\frac{1-\varepsilon_b}{\varepsilon_b}\right) + \ln(K-1)$$

- Thus, in order for α_b to be positive, we only need $\varepsilon_b < 1 - \frac{1}{K}$.

Original paper ([HRZZ09]): T. J. Hastie, S. Rosset, J. Zu, and H. Zou. Multi-class AdaBoost, Statistics and Its Interface, 2009.

Outline

Summary


$\mathbf{w}^{(1)}$
$\mathbf{w}^{(2)}$
$\mathbf{w}^{(B)}$
$\alpha_1 \cdot$
$+\alpha_2 \cdot$
$+$
$+\alpha_{B-1} \cdot$
$+\alpha_B \cdot$

References I

[Bre01] Leo Breiman.
Random forests.
Machine Learning, 45(1):5–32, 2001.

[FS96] Yoav Freund and Robert E. Schapire.
Experiments with a new boosting algorithm.
In *Proceedings of the International Conference on Machine Learning*, pages 148–156, 1996.

[GEW06] P. Geurts, D. Ernst, and L. Wehenkel.
Extremely randomized trees.
Machine Learning, 63(1):3–42, 2006.

[HRZZ09] Trevor J. Hastie, Saharon Rosset, Ji Zhu, and Hui Zou.
Multi-class adaboost.
Statistics and Its Interface, 2:349–360, 2009.

[SSBD14] Shai Shalev-Shwartz and Shai Ben-David.
Understanding Machine Learning — From Theory to Algorithms.
Cambridge University Press, 2014.